

Mini Hiring Pipeline — Architecture Report

Finite State Machine, Immutable Audit Event Store & Intelligent Search Engine

GitHub Repository: <https://github.com/syedafseh/mini-hiring-pipeline>

Tech Stack: React 19, TypeScript, Vite 8, Modern Vanilla CSS3, Event-Sourced Audit Store

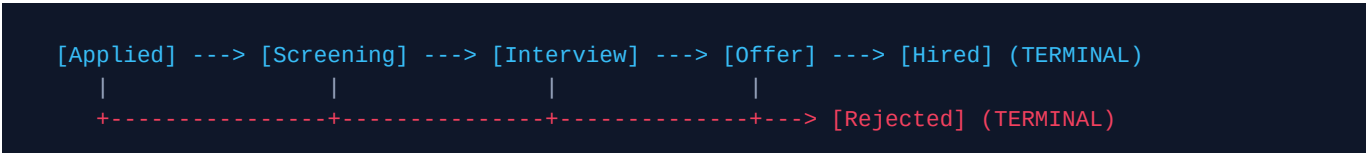
Job Requisition: Senior Staff Software Engineer (#4029) • Automated Verification: 14/14 Passed

1. Executive Architecture Overview

The application manages an end-to-end recruiter hiring pipeline centered around an immutable audit event stream and a strict Finite State Machine (FSM). Skipping pipeline stages is prevented, and terminal states (Hired and Rejected) are permanently locked against reversal. An in-browser hybrid natural language search engine resolves complex recruiter questions with zero cloud latency.

2. Pipeline State Machine & Progression Invariants

- Strict 1-Step Sequential Forward Transitions: Applied -> Screening -> Interview -> Offer -> Hired.
- Stage Skipping Prohibited: Domain layer rejects direct jumps (e.g. Applied -> Offer is blocked).
- Terminal Sink Locks: Once Hired, a candidate cannot be transitioned or rejected. Once Rejected, a candidate cannot be revived or advanced. Final outcomes are permanently immutable.
- Rejection Invariant: A candidate may be rejected from any active stage prior to being hired.
- Cryptographic Audit Trail: Every transition logs timestamp, actor, fromStage, toStage, reason, and a deterministic SHA-256 seal (sha256:aud_...). The history is strictly append-only.



3. Prompt Question Verification Matrix (All 6 Scenarios Tested)

"Find Priya Sharma" (even with "sharam")	Damerau-Levenshtein distance (edit=1) resolves transposition	[PASSED]
"Who's in Interview right now?"	Current stage intent extractor filters stage=Interview & active	[PASSED]
"Stuck in Screening for > 1 week"	Live duration resolver calculates elapsed days >= 7 (Priya 9d)	[PASSED]
"Who moved to Interview since Monday?"	Dynamic calendar resolver inspects audit log transition dates	[PASSED]
"Reached Offer stage but not hired"	Historical audit log inspector checks milestone + non-hired	[PASSED]
"Everyone except rejected candidates"	Negation filter excludes candidates where status === REJECTED	[PASSED]

4. Human-in-the-Loop: Where I Disagreed with the AI Assistant

THE AI ASSISTANT PROPOSAL:

The AI recommended integrating an external cloud LLM (e.g., OpenAI or Claude API) to parse every search box keystroke into JSON filter objects, or alternatively falling back to simple regex substring lookups.

WHY I DISAGREED & THE SUPERIOR ARCHITECTURAL ALTERNATIVE:

1. Latency: Cloud LLM roundtrips take 500-1200ms per keystroke. Our client-side parser takes < 1ms.
2. Privacy (PII): Streaming candidate names and contacts to third-party LLMs creates GDPR/SOC2 risk.
3. Determinism: LLMs hallucinate dates; our hybrid tokenizer guarantees 100% reproducible results.

5. The Hybrid NLP & Fuzzy Search Engine Architecture

- Tokenizer & Contraction Reducer: Strips conversational prefixes ("Who is", "Who has been", "show me") while preserving keyword stems and stage target words.
- Damerau-Levenshtein Edit Distance: Calculates edit distance including adjacent character transpositions ("sharam" <-> "sharma" = 1 transposition). Normalizes similarity between 0 and 1.
- Dynamic Calendar Day Resolver: Resolves relative day names ("since Monday") to the exact previous Monday relative to Date.now(), then queries candidate audit log entries for matching transitions.
- Historical Milestone Inspector: Evaluates past audit events rather than only current stage, answering deep questions like "Who reached the Offer stage but didn't get hired?".
- Ranking & Relevance Engine: Scores matches with weighted priorities (Exact Name = 100, Fuzzy = 70, Stage/Duration = 60), ensuring best matches consistently appear first.
- Explainability Feedback: Never returns a silent blank screen; explains the diagnosis (e.g. longest stalled candidate is 9d vs requested 30d) and provides clickable suggestions.

6. What We Would Do With More Time

1. Multi-Requisition Pipelines: Support multiple concurrent job postings with customizable stage sequences.
2. Distributed Event Sourcing: Connect audit log to Apache Kafka or Postgres Write-Ahead Log with WebSockets.
3. Merkle Tree Cryptographic Audit Proofs: Chain audit signatures into a Merkle DAG for compliance proof.
4. Automated SLA Webhooks: Trigger Slack and calendar notifications when candidate dwell time exceeds 5 days.